

toolc: An Optimizing Compiler for Agent Tool Surfaces

Bryan Healey
bryan@toolc.dev

Draft, August 2026*

Abstract

Language-model agents increasingly act through tools exposed via the Model Context Protocol (MCP). In practice, agents connect to several servers at once, and the naive federation of their catalogs scales poorly: every tool definition occupies context, near-duplicate tools degrade selection accuracy, and multi-step workflows must be improvised call-by-call. We present TOOLC, an optimizing compiler that treats the federated tool surface as a compilation target. TOOLC introspects downstream servers (or synthesizes catalogs directly from OpenAPI specifications and human-readable API documentation) into a capability-graph intermediate representation, then applies a sequence of independently toggleable passes: dead-tool elimination, LLM-assisted description rewriting with intra- and inter-server disambiguation, consolidation of near-duplicate tool families into deterministic facade tools, macro inlining, and an optional retrieval-based selection surface. The compiled artifact is served as a single drop-in MCP endpoint with full call logging, serve-time result compaction, and error-driven re-optimization. In a pilot evaluation over 131 tools from five live servers (30 tasks $\times$ 3 trials, judged answers), the compiled surface with consolidation matched the raw federation’s task success (96.7%) at $6.4\times$ lower cost per completed task, while a consolidation-only configuration reached 100% success at half the raw cost. Every optimization is a measured, reversible configuration choice rather than a fixed architecture.

1 Introduction

Tool use has become the dominant mechanism by which language-model agents act on the world, and the Model Context Protocol has emerged as a standard transport for exposing tools to agents. The unit of deployment, however, is the *server*, not the *task*: a working agent setup typically federates several servers, each contributing tens of tool definitions written by different authors with different conventions.

This federation is an unoptimized program. Its costs are threefold:

1. **Context.** Every served tool definition is prompt input on every turn. In our reference federation, 131 definitions consume roughly 24,000 tokens before the agent reads the user’s request.
2. **Selection accuracy.** Catalogs contain near-duplicates (`search_x`, `find_x`, `get_x`), machine-generated descriptions, and cross-server overlap (two news sources). The agent’s tool choice degrades with catalog entropy, and errors cost round trips.

*Preliminary draft. Evaluation numbers are from a 30-task, 3-trial pilot benchmark and will be superseded by a larger, multi-model study before submission.

3. **Orchestration.** Common multi-step workflows (find an entity, fetch its records, extract a field) are re-derived by the agent on every occurrence, at inference prices.

Our thesis is that these are compiler problems. The tool surface admits an intermediate representation, semantics-preserving transformations over that representation, and a runtime that executes the compiled program while observing it for further optimization. TOOLC realizes this thesis end to end: it is available as an Apache-2.0 engine and as a hosted service that adds managed endpoints, observability, and usage-based billing.

This paper makes four contributions:

1. A capability-graph IR for federated tool surfaces that preserves original definitions as immutable facts and expresses every transformation as a reviewable overlay (§3).
2. A pass architecture comprising dead-tool elimination, LLM-assisted rewriting, deterministic consolidation of tool families into facades (within and across servers), macro inlining, and a retrieval-based selection surface; each pass is independently toggleable and its effect independently measurable (§4).
3. A serving runtime with logged dispatch, serve-time result compaction, and profile-guided re-optimization that converts recurring runtime errors into proposed compiler fixes (§5).
4. A judged, ablated evaluation methodology and pilot results showing that compilation dominates the naive federation on both cost and accuracy axes (§6).

2 Related Work

Tool learning and selection. A substantial literature addresses tool-augmented language models and tool selection at scale, including retrieval over large tool inventories and hierarchical selection. Our selection pass adopts retrieval (BM25 over compiled descriptions) as one point on a measured cost/accuracy frontier rather than as the architecture.

API-to-agent interfaces. Automatic conversion of OpenAPI specifications into agent tools is widely practiced and widely observed to produce poor surfaces; our composition frontend embraces the mechanical conversion precisely because the compiler owns making the result good, and our evaluation includes a deliberately naive machine-generated server as a worst-case input.

Prompt and context optimization. Description rewriting relates to automatic prompt optimization; result compaction relates to context summarization. TOOLC differs in binding these to a persistent, versioned artifact with provenance (original text is never destroyed) and in validating LLM proposals with deterministic checks before they take effect.

Compiler framing. We borrow the classical structure — frontend, IR, passes, emitter, profile-guided optimization — and find it maps cleanly: tool catalogs are modules, near-duplicate families are common subexpressions, macros are inlined procedures, the selection surface is lazy loading, and runtime error clustering is PGO.

3 The Capability-Graph IR

A compiled surface is a graph $G = (S, T, E)$: sources S (downstream servers with catalog hashes), tools T , and data edges E (produced-by / consumed-by relations contributed by macros). Each tool node carries:

- identity (`source:name`), the *original* description and JSON-Schema input schema, immutable once introspected;
- *overlays*: the rewritten description, visibility (`hidden` with a reason), and surface placement (`surfaced`);
- a kind: `passthrough`, `facade`, `macro`, or `meta`;
- for facades, a deterministic routing table from action names to member tool ids.

Two properties matter. **Provenance**: originals are never mutated, so any transformation is auditable and reversible, and user-facing consoles can display exact before/after diffs. **Content addressing**: the graph serializes deterministically and is versioned by content hash, so a served artifact is immutable and cache keys (e.g., for rewrites) are stable.

Frontends produce this IR from three inputs: live MCP introspection, OpenAPI specifications (each operation becomes a passthrough tool with constraints — enums, bounds, defaults — carried into the schema), and human-readable documentation, from which a draft specification is synthesized by an LLM under never-invent rules and then *probe-validated*: parameterless GET endpoints are called against the live API, and endpoints that return 404 are flagged as probable hallucinations before anything compiles.

4 Optimization Passes

Passes run in a fixed order, each mapping graph to graph; every pass emits a structured diff for review. All passes are user-toggable configuration.

Dead-tool elimination hides tools matched by user globs or by built-in noise patterns (health checks, debug endpoints).

Rewrite regenerates descriptions with an LLM, batched per server with full sibling context plus a digest of the rest of the pool, under instructions to disambiguate against both siblings and overlapping tools on other servers, to surface parameter constraints, and never to invent capabilities. Proposals are cached by (tool, original-description hash, prompt version) and can be gated on human review.

Consolidate merges families of near-duplicate tools into *facade* tools with an **action** discriminator. The LLM only proposes group membership and writes the group description; the compiler validates every proposal (members exist, no reuse, size bounds, no id collisions) and synthesizes the union schema and routing table deterministically, so a poor proposal can be rejected but never mis-routed. Dispatch validates arguments against the *member's* original schema. An opt-in second phase merges overlapping capabilities *across* servers (two news providers) into capability facades whose actions name their source, making vendor choice an explicit, documented decision.

Macro inlining adds hand-authored multi-step tools (typed steps over existing tools) as first-class nodes, contributing data edges to the graph.

Selection replaces the served catalog with two meta-tools — `search_tools` (BM25 retrieval over compiled definitions) and `call_tool` (validated dispatch by id) — shrinking served context to a few hundred tokens regardless of catalog size, at the price of a search round trip.

5 Serving Runtime

The compiled artifact is served as a standard MCP endpoint. Every dispatch is logged (arguments, result, latency, parent call for facade/macro/meta fan-out), giving operators full-payload

observability with live tail.

Result compaction. Results exceeding a token threshold are summarized at serve time by a small model under instructions that preserve identifiers and sourcing metadata (record ids, permalinks, timestamps), with a deterministic structural fallback (bounded arrays, capped strings) when the model is unavailable, and an asynchronous mode above a second threshold that serves a structural cut immediately while caching the model-compacted version for repeat calls. Error results and selection meta-results are never compacted.

Profile-guided re-optimization. Recurring runtime errors are clustered by (tool, normalized error signature); each new cluster is analyzed by an LLM against the tool definition *as agents currently see it*, and the proposal is constrained to a typed action: amend the description (a persistent overlay) or block the tool. Operators apply fixes with one action; the applied overlay survives recompilation. The loop was validated by reproducing, automatically, two fixes we had first made by hand (a missing numeric constraint; a chronically failing tool).

6 Evaluation

Setup (pilot). 131 tools across five live servers: a filesystem reference server, the public Hugging Face and GitHub MCP servers, and a deliberately naive machine-generated National Weather Service server (one tool per GET operation, verbatim descriptions). 30 tasks spanning five categories (single-tool, ambiguous-selection, cross-server chains, needle-in-haystack, no-tool-exists), 3 trials each, agent `claude-sonnet-4-6`, answers judged by a stronger model against task-specific rubrics. All conditions traverse the same gateway hop.

Condition	Success	Cost / completed task	Served context (tok)
Raw federation (131 tools)	96.7%	\$0.263	~24,360
Compiled (rewrite+macros+selection)	91.1%	\$0.050	~845
Compiled + consolidation	96.7%	\$0.041	~845
Consolidation, no selection	100.0%	\$0.134	~14,506
Cross-server consolidation	95.6%	\$0.052	~845

Table 1: Pilot results (30 tasks $\times$ 3 trials per condition; 95% CIs overlap for the top three compiled conditions). Preliminary; a larger, multi-model study is in progress.

Three observations. First, *consolidation dominates both ends of the frontier*: it recovers the accuracy the selection surface costs (91.1% $\rightarrow$ 96.7%) while further reducing cost, and without selection it achieves perfect pilot success at half the raw cost — fewer, better-separated definitions improve both direct choice and retrieval. Second, the *selection surface is the cost lever*: served context becomes essentially independent of catalog size. Third, *cross-server consolidation is safe but under-tested here*: the pilot catalog contains minimal genuine cross-server overlap (the pass correctly proposed a single group), so its value case rests on overlap-rich pools and is deferred to the larger study.

Threats to validity. Small task count and trial count (wide intervals); judge circularity (a Claude model judging a Claude agent, mitigated by rubric-based judging of final answers only and by human spot-checks); live-service drift between condition snapshots (mitigated by compiling all conditions from a single catalog snapshot); single agent model (the larger study adds model diversity).

7 Discussion

The compiler framing pays off in governance as much as performance. Because every transformation is an overlay over immutable originals, the system can show its work: consoles render exact diffs, facades display per-action provenance, and operator overrides compose with automated passes rather than fighting them. The same properties make LLM participation safe: models propose (rewrites, groupings, specification drafts, error fixes) while deterministic validators decide, and every acceptance is reviewable and reversible.

The runtime completes a loop that static compilation cannot: logged calls reveal which tools agents actually use and how they fail; failures become typed fix proposals; applied fixes recompile the surface. We believe this observe–optimize–recompile loop, rather than any single pass, is the durable contribution.

8 Future Work

The immediate step is the larger evaluation: more tasks, additional agent models, and an overlap-rich condition that can measure cross-server consolidation’s value. Beyond it: learned (log-driven) dead-tool elimination and macro *mining* from recurring call chains; schema-level rewriting (folding parameter guidance into field descriptions); and compilation targets beyond MCP.

Availability

The engine is open source (Apache-2.0) at <https://github.com/toolc-dev/toolc> and on npm as `@toolc/*`; the hosted service is at <https://toolc.dev>. Benchmark tasks, transcripts, and reports ship with the repository.